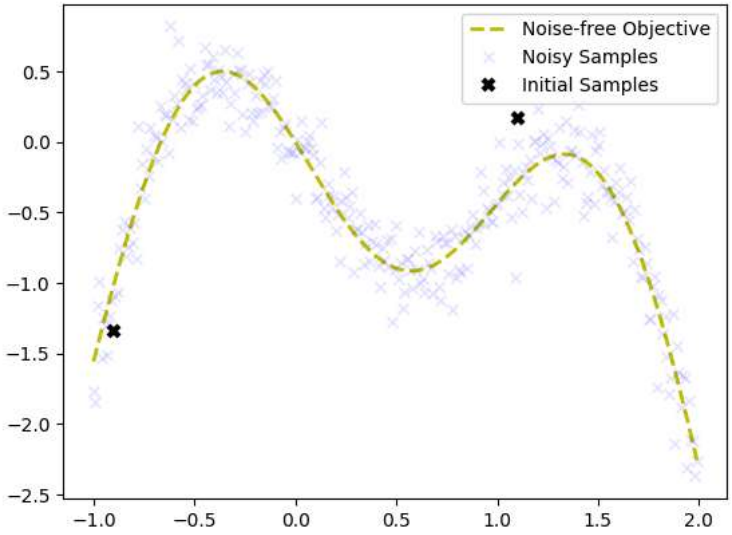
black box function

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

bounds = np.array([[-1.0, 2.0]]) # one parameter with a bound
noise = 0.2

def f(X, noise=noise):
    return -np.sin(3*X) - X**2 + 0.7*X + noise* np.random.randn(*X.shape)

X_init = np.array([[-0.9], [1.1]])
Y_init = f(X_init)
# print(bounds[:, 0])
X = np.arange(bounds[:, 0], bounds[:, 1], 0.01).reshape(-1, 1)
Y = f(X, 0) # without noise
plt.plot(X, Y, 'y--', lw=2, label='Noise-free Objective')
plt.plot(X, f(X), 'bx', lw=1, alpha=0.1, label='Noisy Samples')
plt.plot(X_init, Y_init, 'kx', mew=3, label='Initial Samples')
plt.legend()
```

```
/tmp/ipykernel_6488/3435879221.py:14: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X = np.arange(bounds[:, 0], bounds[:, 1], 0.01).reshape(-1, 1)
<matplotlib.legend.Legend at 0x795fd6cd3920>
```



acquisition function

```
from scipy.stats import norm

def expected_improvement(X, X_sample, Y_sample, gpr, xi=0.01):
    """X is the candidate point I'm considering"""
    mu, sigma = gpr.predict(X, return_std=True) #  The GPR has already been fitted on X_sample before this function i
    mu = mu.reshape(-1, 1)
    sigma = sigma.reshape(-1,1)
    mu_sample = gpr.predict(X_sample)
    # noisy free
    # mu_sample_opt = np.max(mu_sample)
    # noisy function
    mu_sample_opt = np.max(Y_sample)
    with np.errstate(divide='warn'): # aware may have divide by 0
        imp = mu - mu_sample_opt - xi
        Z = imp/sigma
        ei = imp * norm.cdf(Z) + sigma * norm.pdf(Z)
        ei[sigma == 0.0] = 0.0  # sigma = 0 usually mean the point is already sampled and get concrete example
    return ei
```

get next point position

```
from scipy.optimize import minimize # this only do minimize, so maximize -EI
def propose_location(acquisition, X_sample, Y_sample, gpr, bounds, n_restarts=25):
    """
    The acquisition function is non-convex — it has many local maxima (peaks of promising regions). A single optimiza
    The solution: start from 25 random points across the search space, optimize from each, keep the best result.
    """
    dim = X_sample.shape[1]
    min_val = 1
    min_x = None
    def min_obj(X):
        return -acquisition(X.reshape(-1, dim), X_sample, Y_sample, gpr)
    for x0 in np.random.uniform(bounds[:, 0], bounds[:, 1], size=(n_restarts, dim)): # each row is a random point in t
        res = minimize(min_obj, x0=x0, bounds=bounds, method='L-BFGS-B') # get one local max of EI
        if res.fun<min_val:
            min_val=res.fun
            min_x = res.x
    return min_x.reshape(-1,1) # return a array
```

drawing

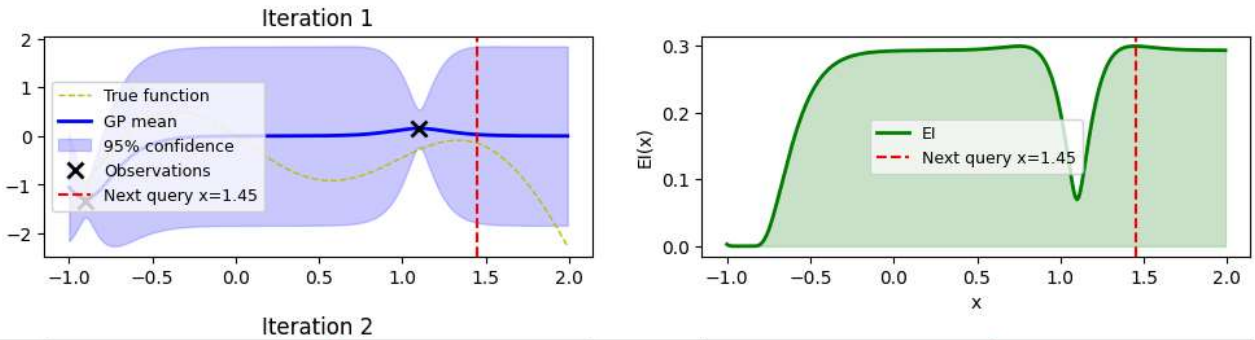
```
def plot_approximation(gpr, X, Y, X_sample, Y_sample, X_next=None, show_legend=True):
        """
        X              : dense grid of candidate points, shape (n, 1)
        Y              : true function values at X (if known), shape (n, 1)
        X_sample: observed points so far, shape (m, 1)
        Y_sample: observed values so far, shape (m, 1)
        X_next    : next proposed point (optional), shape (1, 1)
        """
        mu, std = gpr.predict(X, return_std=True)
        mu = mu.flatten()
        std = std.flatten()

        plt.plot(X, Y, 'y--', lw=1, label='True function')
        plt.plot(X, mu, 'b-', lw=2, label='GP mean')
        plt.fill_between(X.flatten(),
                                    mu - 1.96 * std,
                                    mu + 1.96 * std,
                                    alpha=0.2, color='blue', label='95% confidence')
        plt.plot(X_sample, Y_sample, 'kx', mew=2, markersize=8, label='Observations')
        if X_next is not None:
                plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
        if show_legend:
                plt.legend(fontsize=9)

def plot_acquisition(X, X_sample, Y_sample, gpr, xi=0.01, show_legend=True):
        """
        X              : dense grid of candidate points, shape (n, 1)
        X_sample: observed points so far, shape (m, 1)
        Y_sample: observed values so far, shape (m, 1)
        gpr          : fitted GaussianProcessRegressor
        """
        ei = expected_improvement(X, X_sample, Y_sample, gpr, xi=xi)
        ei = ei.flatten()

        X_next = float(X[np.argmax(ei)])

        plt.plot(X, ei, 'g-', lw=2, label='EI')
        plt.fill_between(X.flatten(), 0, ei, alpha=0.2, color='green')
        plt.axvline(x=X_next, ls='--', c='red', lw=1.5,
                                label=f'Next query x={X_next:.2f}')
        plt.xlabel('x')
        plt.ylabel('EI(x)')
        plt.legend(fontsize=9)
        if show_legend:
            plt.legend(fontsize=9)
```

full BO loop

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import ConstantKernel, Matern

m52 = ConstantKernel(1.0)*Matern(length_scale=1.0, nu=2.5)
gpr = GaussianProcessRegressor(kernel=m52, alpha=noise**2)
X_sample = X_init
Y_sample = Y_init
n_iter = 10
```

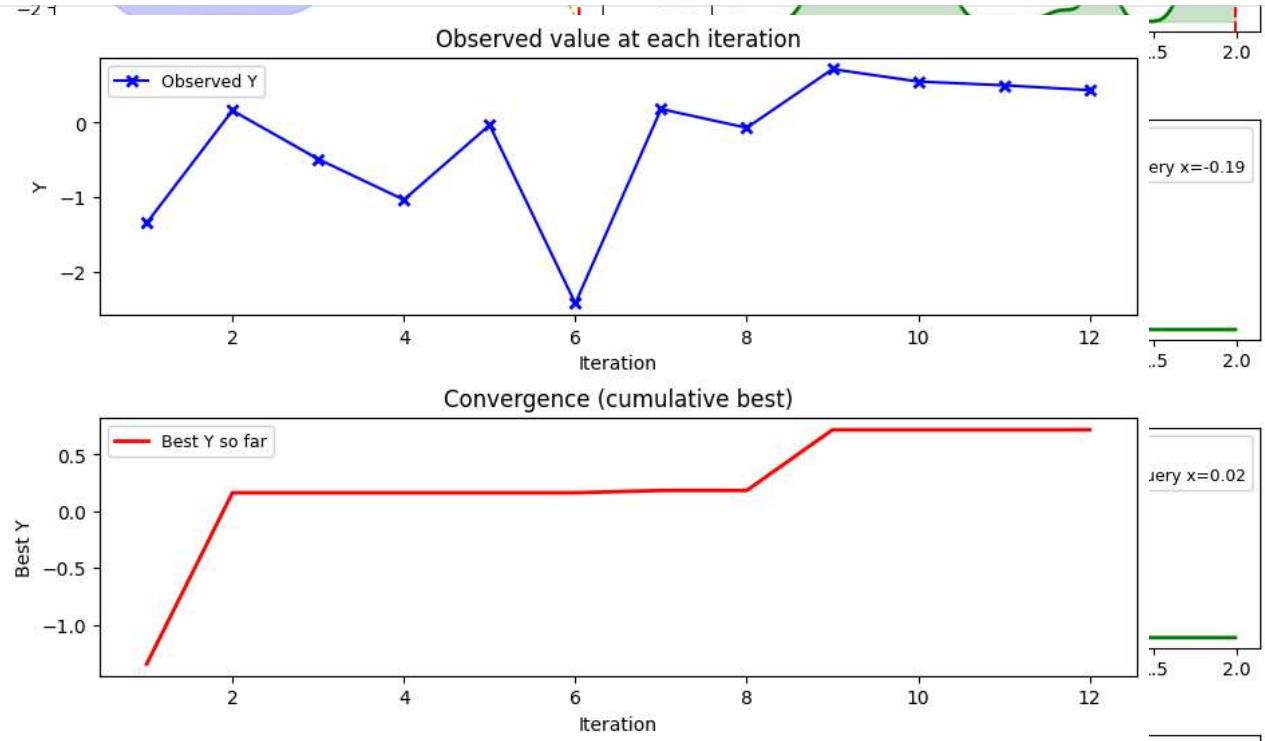
```
plt.figure(figsize=(12, n_iter*3))  # 3 inch each
plt.subplots_adjust(hspace=0.4)
for i in range(n_iter):
    gpr.fit(X_sample, Y_sample)
    X_next = propose_location(expected_improvement, X_sample, Y_sample, gpr, bounds)
    Y_next = f(X_next, noise)
    plt.subplot(n_iter, 2, 2*i+1)
    plot_approximation(gpr, X, Y, X_sample, Y_sample, X_next, show_legend=i==0)
    plt.title(f'Iteration {i+1}')
    plt.subplot(n_iter, 2, 2*i+2)
    plot_acquisition(X, X_sample, Y_sample, gpr, show_legend=i==0)
    X_sample = np.vstack((X_sample, X_next))
    Y_sample = np.vstack((Y_sample, Y_next)) # append the new X into original
```

```
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
/tmp/ipykernel_6488/1113484212.py:21: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  plt.axvline(x=float(X_next), ls='--', c='red', lw=1.5, label=f'Next query x={X_next[0,0]:.2f}')
/tmp/ipykernel_6488/1113484212.py:35: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
  X_next = float(X[np.argmax(ei)])
```

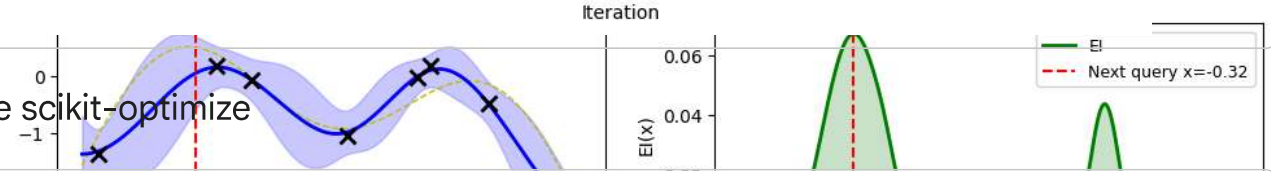



```
def plot_convergence(X_sample, Y_sample):
    """
    X_sample: all observed inputs, shape (n, 1)
    Y_sample: all observed outputs, shape (n, 1)
    """
    x = X_sample.flatten()
    y = Y_sample.flatten()
    n = len(x)
    best_so_far = np.maximum.accumulate(y)  # cumulative max at each step

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6))
    plt.subplots_adjust(hspace=0.4)
```

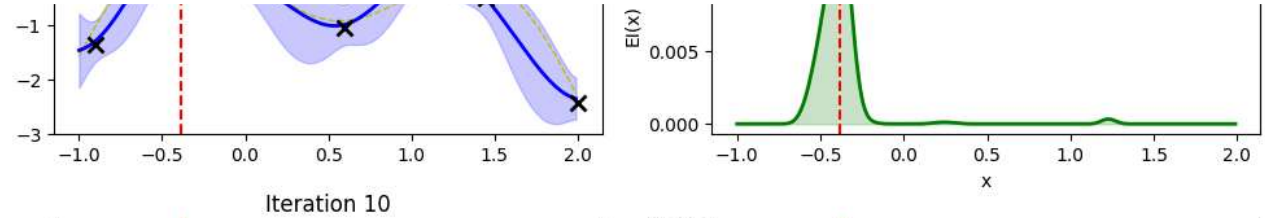
```
        # top: raw observations each iteration
        ax1.plot(range(1, n+1), y, 'bx-', mew=2, label='Observed Y')
        ax1.set_xlabel('Iteration')
        ax1.set_ylabel('Y')
        ax1.set_title('Observed value at each iteration')
        ax1.legend(fontsize=9)

        # bottom: best found so far
        ax2.plot(range(1, n+1), best_so_far, 'r-', lw=2, label='Best Y so far')
        ax2.set_xlabel('Iteration')
        ax2.set_ylabel('Best Y')
        ax2.set_title('Convergence (cumulative best)')
        ax2.legend(fontsize=9)

        plt.show()

plot_convergence(X_sample, Y_sample)
```

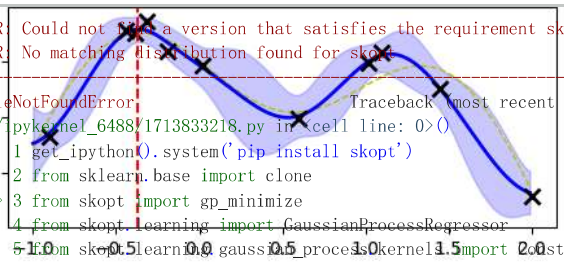


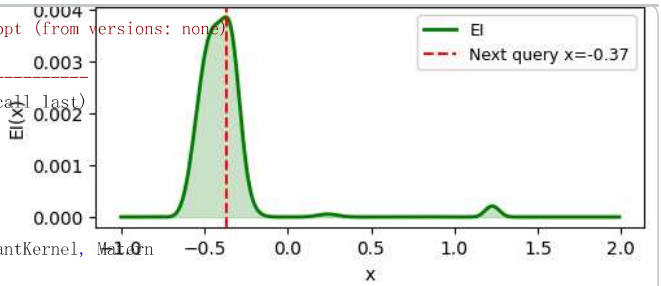
use scikit-optimize




```
!pip install skopt
from sklearn.base import clone
from skopt import gp_minimize
from skopt.learning import GaussianProcessRegressor
from skopt.learning.gaussian_process.kernels import ConstantKernel, Matern

m52 = ConstantKernel(1.0) * Matern(length_scale=1.0, nu=2.5)
gpr = GaussianProcessRegressor(kernel=m52, alpha = noise**2)
r = gp_minimize(lambda x: -f(np.array(x))[0],
                bounds.tolist(),
                base_estimator=gpr,
                acq_func='EI',
                xi=0.01,
                n_calls=10, # number of iterations
                n_randome_starts=0,
                x0=X_init.tolist(),
                y0=-Y_init.ravel()
                )
gpr.fit(r.x_iters, -r.func_vals)
plot_approximation(gpr, X, Y, r.x_iters, -r.func_vals, show_legend=True)
```








```
ERROR: Could not find a version that satisfies the requirement skopt (from versions: none)
ERROR: No matching distribution found for skopt
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_6488/1713833218.py in <cell line: 0>()
      1 get_ipython().system('pip install skopt')
      2 from sklearn.base import clone
----> 3 from skopt import gp_minimize
      4 from skopt.learning import GaussianProcessRegressor
      5 from skopt.learning.gaussian_process.kernels import ConstantKernel, Matern

ModuleNotFoundError: No module named 'skopt'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES